SIT310: Lab Sheet - ROS Turtlesim Teleop

Last Updated: 3 March 2026 v1.1

Introduction

This task will introduce you to the software tools we will use in this unit. Most of the commands are specified for you in the linked instructions or below. At the end of this task, you should have a working system for future assignments and a basic understanding of ROS topics and nodes. You **won't** be using the Duckiebot in this lab exercise, so can keep it turned off. All commands are to be run using your Raspberry Pi development machine. This lab sheet could be done over two class sessions. We will use Docker to run the Duckietown ROS package in a container.

It is suggested that you review the Nodes, Messages and Topics sections in the link below as private study:

https://wiki.ros.org/ROS/Concepts

Setup Prerequisites

For on-campus students:

1. Create a group of 2 in class to work together. You can work alone later for your tasks, however you will need a version of Ubuntu installed – either your own Raspberry Pi, or a Ubuntu PC or Ubuntu running on a virtual machine on your Windows/Mac PC.
2. For on campus students, each group will be supplied with a box containing the Duckiebot and a Raspberry Pi. The Raspberry Pi (Model 4) with an SD card runs the Ubuntu 22.04.x LTS version. Take note of your box QR code and Raspberry Pi number as this will be your group's development PC throughout the unit, and you should use the **SAME** Raspberry Pi as your configurations are saved on the Pi's SD card. You will need to create a user account for your Raspberry Pi Ubuntu, so select a username and password and do the following.
   i) Login to the Raspberry Pi with the default user: Ubuntu password: sit310.
   ii) Go to Settings -> Users. Click Unlock at the top right corner. Click "Add User" and fill in your selected details.
   iii) Logout from Ubuntu. Now select your user profile and login to Ubuntu again.

iv) Check that your Pi can connect to RIOT Lab hotspot and can access the Internet. If there are issues with connection, then you will need to use your mobile phone and setup a hotspot and connect to the hotspot for Internet access. Let your tutor know so we can update the Pis.

3. You are now ready to setup your Raspberry Pi with your Ubuntu profile for use with Duckietown, Docker, GitHub and ROS.
   I. Setup Laptop (your Raspberry Pi)=> https://docs.duckietown.com/daffy/opmanual-duckiebot/setup/setup_laptop/index.html. Note: Use *daffy* version for Duckietown Shell (dts) installation. 'ente' is the latest version, but we have not tested it sufficiently with Raspberry Pi.
   II. Setup accounts => https://docs.duckietown.com/daffy/opmanual-duckiebot/setup/setup_account/index.html

For online students:

Install Ubuntu for your PC or run Ubuntu (22.4.x) on a virtual machine. Once installed, you will need to setup your computer following the steps here:

i. Setup laptop/PC=> https://docs.duckietown.com/daffy/opmanual-duckiebot/setup/setup_laptop/index.html. Note: Use *daffy* version for Duckietown Shell (dts) installation. 'ente' is the latest version, but we have not tested it sufficiently with Raspberry Pi.
ii. Setup accounts => https://docs.duckietown.com/daffy/opmanual-duckiebot/setup/setup_account/index.html

It is suggested that you check in with the tutor when you reach this stage before proceeding. The setup will take a good amount of time. We suggest browsing to learn more about ROS while waiting for packages to install.

Setup Github with Duckietown repository template

We will use a template to create a new Github repository to hold the code we are going to create during this unit. Ensure that you have completed the previous steps and setup your Duckietown, Docker and Git as per above.

You will need to have a Github account with a personal access token.

Create a Github Account

Open https://github.com/ and Sign Up to create a new account, or Sign In if you already have an existing account. Create a Personal Access Token if you haven't done so before and use it for the following. Note: All Github access or changes will need this token as the password.

Create a Personal Access Token (if you haven't done so previously)

Since we will be working at the command line, we will need a personal access token instead of a password for Github. To set one up, access your user profile using the avatar button and click on Settings.

At the bottom of the screen, click on Developer Settings. Select Personal Access Tokens on the left, then Tokens (classic).

Click on Generate a New Token, then Generate New Token (Classic). You may need to authenticate with 2FA.

Enter a name for the token eg Duckietown.
Set the expiry date to Custom and enter a date after the end of trimester and the marking period eg 1 year into the future.
In the permissions section, click the Repo checkbox to give Full control of private repositories
Now click Generate Token at the bottom of the page.

In the subsequent page, you will see your new token. Copy it and save it somewhere safe, as when you close the browser you will not be able to see it again. If you lose it, you will need to repeat these steps to generate a new token.

Fork the Template Repository

Now open the repository we will use as a template at https://github.com/UML-EECE-5560/eece5560-base and click Use this Template and select Create a New Repository.

Since you are already signed into Github, it will create a repository based on this template for your account. Enter a name for the repository that only uses lowercase letters, numbers and hyphens. Otherwise the docker commands in later tasks will fail.

Also remember to make the repository PRIVATE to prevent other students from copying your code. If you are working in a team, share the repository with your team mate.

Clone your Repository on the Raspberry Pi

Open a terminal on the Raspberry Pi with Ctrl+Alt+T and enter the following command, replacing GITHUBNAME and REPONAME with the name of your repository

```
git clone https://github.com/GITHUBNAME/REPONAME.git
```

This will create a local copy of your forked repository on the Raspberry Pi.

Now change directory into your repository using

```
cd REPONAME
```

Updating the Dockerfile

The repository allows the code you write to be packaged into a Docker container and run on your robot. While its configuration in the Dockerfile is mostly set up, there are two changes you will need to make.

Edit the Dockerfile with nano or gedit editor (or use the Ubuntu File Manager and Text Editor):

nano Dockerfile

or

gedit Dockerfile

On the second line, you will see

ARG REPO_NAME="eece-base"

Change this to the name of your repository eg if your repository was sit310, you should have

ARG REPO_NAME="sit310"

On the fourth line, you will see

ARG MAINTAINER="Paul Robinette (paul_robinette@uml.edu)"

Update this with your name and email address

Press Ctrl-X to exit, Y to save the file, and Enter to keep the same name.

Committing the Changes to Github

Your changes only exist in your local repository on the Raspberry Pi, not in the forked repository on Github.

First you can see the files that have been changed using

```
git status
```

In this case we updated the Dockerfile, so you should see it listed as modified.
Next you need to inform git that you want to add the changes to the file with your next commit. Use

```
git add .
```

to add all files under the current directory.

You can double check which files are staged for inclusion in the next commit by running git status again.
Now you can commit the changes, which generates a new state of your repository along with a message so you know what was changed and why.

```
git commit -m "Updated Dockerfile"
```

In your own commits in future, you can be more descriptive about which features you created, or bugs you fixed with this commit. This allows you to jump back and forwards in time if you find something didn't work as you expected.

Now the local repository has stored a snapshot of the current state with the changes, but they still only exist on the local raspberry Pi. We can push them to the forked repository in Github using

```
git push
```

If this is your first commit with this branch, you may need to use the longer form

```
git push origin master
```

Using the Repository in Future Tasks

In future tasks, when you use dts devel build or dts devel attach commands, you should always run them from within your repository, so when you open a new terminal on the Raspberry Pi, remember to do the cd REPONAME before you run your commands.

Also ensure that you have always committed all changes in your repository before running the dts devel commands. You can check for changes with git status, add all files with git add . and then commit all changes with a message using git commit -m "Updated the code to ..."

Lab Sheet Activity

1. Building a docker image to use Duckietown ROS image

1. **Make sure you are in your Git repo directory in Ubuntu**, then use dts to build your docker image:

   dts devel build --pull

   This command will build your Dockerfile into an image that you can run as needed. It needs to be run when there are big changes, like when you update the template to your repo or when some Duckietown images are changed. The “pull” option tells dts to check for any updates to other dependency images when it runs.

   [Pro Tip 1]: If you are getting the error “The command \bin\bash -c dt-apt-install ${REPO_PATH}/dependencies-apt.txt’ returned a non-zero code, the **REPO_NAME** argument in your **Dockerfile** is wrong. Change it to match the name of the git repo name.

   [Pro Tip 2]: You will notice that if there are any uncommitted changes, this command will complain “Your index is not clean (some files are not committed)”. You will need to commit your changes to your repo using git. Remember: you can check uncommitted changes using git status, stage to commit using git add your_file, commit them using git commit -m “comment for this commit” and push them with git push.

   [Pro Tip 3]: This command may take several minutes, especially at step 10/57, the first time it runs. You can first download the biggest pre-requisite docker image by running the command below.
   docker pull duckietown/dt-core:daffy-amd64

[**Update 2026 Pro Tip 4]** How to Fix expired security key error at dts devel build –pull

an expired GPG security key.
An EXPKEYSIG F42ED6FBAB17C654 Open Robotics means the security "handshake" between your Raspberry Pi and the ROS package servers has expired.

Solution:

i. Open your Dockerfile with a text editor or CLI editor e.g. sudo nano Dockerfile

ii. Look for the line that says

FROM duckietown/${BASE_IMAGE}:${BASE_TAG} as BASE.

iii. **Immediately after that line**, add this:

RUN apt-key adv --keyserver 'hkp://keyserver.ubuntu.com:80' --recv-key C1CF6E31E6BADE8868B172B4F42ED6FBAB17C654

iv. Save and Exit (Ctrl+O, Enter, Ctrl+X).

Although you clicked save, the Duckietown Shell (dts) has not "saved" that change to your Git history yet. Follow these steps in terminal:
Bash

```
git add Dockerfile
git commit -m "Added ROS GPG key fix"
git push
```

v. Run the build again: dts devel build –pull, and it should work successfully.

2. You can now run your image as a docker container using dts:

```
dts devel run -M -X --cmd bash
```

Let's take a deeper look at this command:

- **dts devel run** command starts a Duckietown-based container (like what is in the class repo you used as a template) with a configuration that lets it run like Docker images that will run on your robot. This saves us from writing a very long docker run command with many options.
- The **-M** option mounts the current project to the working directory of the docker container. This allows us to write code outside of the container and run it inside it, which can be handy depending on which text editor you prefer. It also lets us use git to store code as we write it. This will be our method for future tasks.
- The **-X** option allows us to run graphical interfaces.

- The **--cmd** option tells dts to run a bash shell (likely the same command line terminal as you are using) after starting the container so that you can type commands.

4. If you run **ls** command inside this container you should see the contents of your repo here. These files are shared with your computer right now, so when you make changes on your computer they will also be changed in docker and vice versa.

2. Building a ROS Environment

5. We need to build the current ROS environment inside this container. Run:

```
cd /code/catkin_ws/
source /code/catkin_ws/devel/setup.bash
catkin build
source /code/catkin_ws/devel/setup.bash
```

These commands allow you to use the correct environment for ROS and build all packages in that environment. You have to run these commands every time you create a new package. For this exercise, once is enough.

6. First, let's start roscore:

```
roscore
```

If roscore runs properly, you will see "started core service [/rosout]" as the last line. As you noticed, roscore is hogging this terminal, so you can't do anything else in this terminal anymore until you kill roscore.

[Pro Tip]: If you want to kill a process running in the terminal, the best practice is to kill it with CTRL+C and wait until the process is dead. If you close a terminal by pressing the "X" button with your mouse, it is possible that the process still runs in the background. This may introduce very different problems. If you suspect that this happened to you, I suggest that you restart Ubuntu. Restarting is the ultimate Pro Tip.

7. Let's start another terminal and connect it with your docker container. First, start a new terminal window or tab, then change the directory to your git repo and run:

[Pro Tip]: CTRL+SHIFT+T shortcut will start a new tab.

```
dts devel run attach
```

This will give you a new command line interface to your running docker container.

8. You need to source your environment in this new terminal so that we can use all of the ROS-specific commands:

```
source /code/catkin_ws/devel/setup.bash
```

[Pro Tip]: Use TAB completion when you are navigating in the command line.

3. Running the Turtlesim ROS node

9. I suggest that you follow the tutorial below for running the Turtlesim and exploring ROS.

   https://www.youtube.com/watch?v=rIPmFy_Ax2A

   Time to start a ROS node!

```
rosrun turtlesim turtlesim_node
```

10. You should see a blue window with a turtle in the center. This command will also consume your terminal, so let's start up a third terminal (Step 7) and source our environment (step 8).

    In this terminal, let's use ROS tools to see what is running:

```
rosnode list
```

You should see the "/turtlesim" node here.

The following command will give you more information about the node:

```
rosnode info <node_name>
```

The most important fields in the output are the Publications and Subscriptions. You will see that this node subs to 1 topic and pubs 3 topics.

11. Run:

```
rostopic list
```

This shows all active topics. Two topics, "/rosout" and "/rosout_agg" are always there for internal processes and we can safely ignore them for our purposes. We will focus on the topics that are part of the turtlesim node. These topics should have the prefix "/turtle1/".

12. Let's figure out what is going on with this ROS node. Use the rostopic command for all turtlesim topics.

    rostopic info <topic_name>

    [Pro Tip]: TAB completion works for ROS commands as well and is often very useful.

    Run the **rostopic info** command on the three active topics. One topic will say that it has a subscriber but no publisher. This means that the ROS node /turtlesim/sim is listening (subscribing) to this topic. The other two topics will have a publisher but no subscriber. This means /turtlesim/sim is talking (publishing) over them.

13. We can listen to the output of the ROS node using a command line tool. This is very useful for debugging. Run this command:

    rostopic echo <topic>

    For one of the topics that you think is being sent by /turtlesim node. Leave this command running. You will see streaming output but the values does not change. Let's change that!

14. Let's run a second ROS node! Start a new terminal. Make sure you run dts devel run attach followed by source /code/catkin_ws/devel/setup.bash command in this new terminal. Don't kill the first turtlesim node. If you killed it already, re-start it.

    Run the following command:

    rosrun turtlesim turtle_teleop_key

    You can now move the turtle by using the arrow keys.

15. Running rosnode list command in a new terminal should show the that there are now 2 active nodes (apart from /rosout which is always there).

16. You can also see these nodes visually by running the command below:

    rqt_graph

You may need to choose the "Nodes/Topics (all)" option for it to show the ROS nodes visually.

17. You can end all of the above processes (CTRL+C) and quit any docker containers by typing exit in each of the terminal windows.

Troubleshooting

- In step 5, if the catkin build command is not successful because of duplications in ROS packages with the same name in your directory, backup all of the packages folder at ‘<your_directory/eece5560/’ outside and remove all packages at ‘<your_directory/eece5560/’ and ‘<your_directory/packages/’

- In step 10, if the command returns an error: ‘rosnode not found’, on the new terminal, source bashrc, and then run ‘dts devel run attach’

- If at any point you need to reset your Turtlesim window, do the following: In a new ubuntu terminal, in your repo directory, type, ‘dts devel run attach’, then type ‘source /code/catkin_ws/devel/setup.bash’, then type ‘rosservice list’, here you can see all the different commands to call to run for the turtlesim. To ‘reset the turtle in the turtlesim, type ‘rosservice call /turtlesim/reset’. On the turtlesim window, it should look like it was freshly loaded.